Intent-aware, self-healing GUI automation for accessible AI assistants

MILESTONE 1 – MVP PROPOSAL REPORT (SOFTWARE TRACK)

Authors:
Sharique Baig [**TODO:** ERP]
Maira Aijaz [**TODO:** ERP]
Barkhi Khan [**TODO:** ERP]

Advisors:
[**TODO:** internal advisor]
[**TODO:** co-advisor(s), if any]

[**TODO:** submission date]

Department of Computer Science



School of Mathematics and Computer Science

Contents

1 Summary **2**

2 Problem and validation **2**
2.1 The problem . . . 2
2.2 Alternatives and competitor analysis . . . 2
2.3 Stakeholder evidence . . . 2

3 Approach **2**

4 Technical feasibility demonstrator **3**
4.1 What was built . . . 4
4.2 Why the changes are applied . . . 4
4.3 Evidence so far . . . 4
4.4 Honest limits of the MVP . . . 4

5 Complexity (Seoul Accord D.4.1) **4**

6 Financial and regulatory feasibility **4**
6.1 Cost . . . 4
6.2 Legal and regulatory constraints, and how we stay inside them . . . 5

7 Scope fixed in writing **6**
7.1 In scope . . . 6
7.2 Out of scope . . . 6
7.3 Deliverables per milestone . . . 7

8 AI tools used **7**

9 Team contributions and advisor endorsement **7**

1 Summary

Some people cannot operate a screen themselves, so an AI assistant does it for them. Apps change constantly (a button is renamed, reworded or moved), and an assistant that was built on fixed selectors then breaks or, worse, taps the wrong control *without saying so*. We build a **GUI tool** that an assistant calls with a *goal* ("post a photo with caption hello") instead of a locator. The tool finds the control through a ladder of routes, checks the next screen to confirm that the step really happened, and, when nothing works, stops and tells the user in plain words that they may need to do the step themselves.

What this project is, and is not. The contribution is the tool. The assistants that use it (Kenna, Momo and similar agents) belong to a family we call *synthies*: long-running, permission-scoped agents with an audit trail. The tool is one item in such an assistant's tool list, the one it reaches for when a target has only a screen and no API. The project is an **accessibility** project. It is not an adversarial one: the "breaks" described below are controlled UI changes of the kind every real app ships, used to show that the tool keeps working when the interface drifts, including in a deliberately worst-case combination.

Status at M1. The MVP runs end to end on a mock social-media web app (nine intents) and on stock Android Settings (an emulator), with a real language model as an optional, verified helper. Section 4 gives the evidence and its limits.

2 Problem and validation

2.1 The problem

- Assistive use of GUIs depends on controls having stable, readable identities. Apps break this on every release: renamed ids, reworded copy, moved controls, icons without labels.
- Script-based automation (recorded locators) fails *loudly* when this happens. Model-driven agents fail *quietly*: they act on their first guess, do not check, and report success.
- A user who cannot take over a step needs a third behaviour: heal when it is safe to, verify every step, and otherwise say honestly that it could not finish.

2.2 Alternatives and competitor analysis

Table 1 places the tool among existing approaches. Rows are described at the level of what the approach is designed for; [**TODO:** verify each row against vendor documentation and cite it before submission].

2.3 Stakeholder evidence

[**TODO:** The jury form asks for competitor analysis, stakeholder interviews or a user survey. Competitor analysis is above. No interviews or survey have been done yet. Either run a short set (for example 3–5 interviews with screen-reader or switch-access users, or a 10-question survey) before submission, or state plainly that this is planned for M2 and why. Do not describe any user as having tested the tool unless they did.]

3 Approach

1. **Goal, not locator.** The assistant passes a goal (or the user's own words) to the tool. A plain-English front end maps it to one of nine intents and asks, or declines, rather than

Approach	Designed for	Gap this tool addresses
Selenium / Appium	A developer's test script with hard-coded locators	Breaks on any locator change; "no exception" is treated as success; no notion of a user goal
Healenium, Testim	Healing a test script by re-ranking element attributes	Heals the script's locator, not an intent; no check that the right control was acted on; no plain-language stop
MCP browser tools plus an LLM	A model drives the page step by step from an accessibility snapshot	Model call on every step; the model's own claim of success is trusted; page text goes into the model's context
Screen readers, Voice Access, TalkBack	Direct control by the user, on the user's device	The user must still find and operate every control; no goal-level delegation
Screenshot-and-click agents	General computer use	Strong at clicking; no per-step verification contract, no honest-stop contract for an assistive user

Table 1: Alternatives. Comparison of mechanism, not a measured benchmark of competitors.

guessing.

2. **A ladder of routes.** The tool tries, in order, id, label, text, scoped container, remembered route, model, and position. A route counts only if it points at exactly one control.

3. **Verify on the next screen.** Each step has a success predicate evaluated on the following screen. A tap that changed nothing counts as wrong and the next route is tried. "No exception" is never success.

4. **The model is a helper, never a boss.** It may only answer with indices into the list of controls actually on screen. Its answer is acted on as a candidate and judged by the same predicate. An irreversible intent (for example publishing) needs confidence ≥ 0.8 and uses only the first choice.

5. **Honest stop.** If no route works, the tool stops and returns a first-person message that ends with "You may need to do this step yourself." The assistant relays it unchanged; it does not retry in a loop or offer a cheerier summary.

6. **Memory and teaching.** A model answer that passed verification is remembered. A human can teach a new routine once; the tool learns the steps and a success check for each, and refuses steps it cannot verify. It never records passwords.

Architecture: a harness (intents, ladder, drift injection, scoring) served to a browser by a Node runner, a model bridge (offline stand-in, Anthropic, OpenRouter), and an Android layer that applies the same ladder to UiAutomator trees read over `adb`.

4 Technical feasibility demonstrator

The hardest part of the idea is not clicking; it is *acting correctly when the interface is not what the script expected, and knowing when it did not work.* The demonstrator targets exactly that.

4.1 What was built

- Nine intents for a create-post flow on a mock social app, with scoring and a screen-reader narration region.
- Eleven controlled UI changes (B1–B11), used only as stress tests: renamed ids, stripped labels, reversed layout, duplicated control, nested scroller, bare icon, overlay, hidden control, reworded copy, relocated control, and text inside labels that tries to instruct the model.
- Five arms for comparison: scripted baseline, rules only (healed), model only, model plus check, and hybrid (rules, then model, all verified).
- An Android layer with the same ladder, run live on an emulator against stock Settings.
- Reporting that produces tables with 95% Wilson intervals from raw logs.

4.2 Why the changes are applied

The changes are not attacks on any product. They reproduce what ordinary releases do to a UI: A/B tests reword labels, redesigns move controls, framework upgrades regenerate class names, localisation changes every string, and component libraries sometimes drop accessible names. The extreme combinations answer one question for an assistive user: *what happens in the worst case?* The answer we want is "it still works, or it stops and says so," never "it silently did the wrong thing." The planted-instruction change (B11) is a safety test: an assistant for a person who cannot check the screen must not obey text embedded in a page.

4.3 Evidence so far

Numbers below are copied from `Kenna/results/published/`; [**TODO:** regenerate before submission and re-read each sentence next to its number]. All model results are one model (`qwen/qwen3.7-flash`) via OpenRouter, on one mock app or stock Android Settings. Repetitions use temperature 0 and a cache, so they measure variation in the app and harness, not sampling variation of the model.

4.4 Honest limits of the MVP

One mock web app and stock Android Settings; one real model; hand-written success predicates; no scrolling; no user study with screen-reader users; the Android drift is language and font scale, not an app update. On Android Settings (about nine controls on screen) a bare model matches the hybrid, so verification shows its benefit on the busier web app and with planted instructions, not there.

5 Complexity (Seoul Accord D.4.1)

6 Financial and regulatory feasibility

6.1 Cost

The tool needs Node 18 or later, no GPU and no model key for the offline mode. The real-model qualification cost about $0.02 on OpenRouter (`qwen/qwen3.7-flash`, roughly $0.03 and $0.13 per million input and output tokens); free-tier accounts are capped at 50 requests per

Claim	Result	Source
C1: a recorded script breaks under UI change; the ladder survives it	N=10 per cell, 12 configurations, no model. Ladder completes 100% [72–100] on clean, B1–B7 singly and B1+B2+B3+B4. Recorded script 0% [0–28] on B1, B2, B6, B7 and every combination. Both stop honestly on B8	`rules_full_n10`
C2: a model rung heals what rules cannot read	Rules alone 0% [0–28] on three drifted configurations. Hybrid 100% [72–100] on clean, B9, B1+B9 and B1+B10+B9. Both stop on the most extreme configuration	`semantic_n10_qwen37flash`
C3: verification prevents silent wrong actions	Bare model 0% on every configuration, 1.0–4.4 silent wrong actions per run (on the clean app it silently skips one step). Hybrid: 0 silent wrong actions in every configuration	same report
C6: real drift on Android (language, font)	French: rules 2/10 [6–51]%, hybrid 10/10 [72–100]%. Large font: all arms 8/10, two rows below the fold, nothing scrolls yet	`android_qual_qwen37flash`
C7: how many controls have no name	2% nameless across 5 Settings screens × 3 sessions, once the labels inside a clickable row count (own attributes alone say 78%)	`path-b/dumps/sessions/`
C4/C5: planted instructions; remembered answers	Built and unit-tested. C4 has one real-model observation (n=1); the N=10 runs are pending	`docs/NEXT_STEPS.md`

Table 2: Evidence at M1. Interval values are the published ones; pending items are labelled pending.

day, so the full protocol uses a small paid balance. [**TODO:** add a projected cost for M2–M4 runs and hosting]

6.2 Legal and regulatory constraints, and how we stay inside them

- **Where it is tested.** Only on software we own or are free to test: our own mock web app and stock Android Settings on a local emulator. We do not automate third-party commercial sites, because their terms of service commonly forbid automated access.
- **Bot protection.** We do not evade it. A measure-only probe records how a site's protection layer treats the tool and reports the result. It never bypasses, and nothing in the milestones depends on it. Sites that block automation are outside the scope, and we say so.
- **Real apps later.** Any move to a real third-party app (M4 real-conditions validation) will be read-only or supervised, on the user's own device and account, at human speed, after reading that app's terms, and ideally with the owner's permission. The target will be chosen with the advisor. [**TODO:** decide the M4 target; self-hosted marketplace or open-source apps are the low-risk options]
- **Credentials and data.** No credentials in the repository or in output; a live login lives only in an encrypted local vault set by the human. No user data is collected at M1. [**TODO:** if interviews or a survey are run, state consent and anonymisation]
- **Safety of delegated actions.** Irreversible actions need confirmation and a confidence

Attribute	Where it appears
Depth of analysis (no obvious solution)	Neither fixed locators nor a bare model works: the first breaks, the second makes silent errors. The ladder-plus-verification design came out of measuring both
Depth of knowledge	Accessibility trees on web and Android, UiAutomator parsing, model prompting with a restricted answer space, statistical reporting
Infrequently encountered issues	Windows suppressed from the accessibility tree, overlays, controls nested in clickable rows, actions that change nothing
Outside standard practice	Success is a predicate on the next screen rather than "no exception"; a model is admitted only as a verified candidate
Diverse stakeholders	Assistive-technology users, the assistants that call the tool, app developers whose UIs change [**TODO:** confirm and add evidence]
Significant consequences	A wrong action on someone's behalf (for example publishing) is hard to undo; this is why irreversible intents are gated
Interdependent sub-problems	Intent parsing, route ladder, model bridge, verification, memory, Android layer, reporting
Ill-defined requirement	"Works when the UI changes" had to be turned into measurable configurations and metrics

Table 3: Self-assessment against the eight attributes. The jury scores; this table says where we believe each applies.

threshold; planted instructions in page text are treated as untrusted content.

7 Scope fixed in writing

7.1 In scope

- The GUI tool: intents, route ladder, step verification, honest-stop messages, model rung, route memory, teaching.
- Evaluation on the mock web app (B1–B11 and combinations) and stock Android Settings, with one real model at M1.
- Reproducible reports (manifest, tables, raw logs) for every claim.

7.2 Out of scope

- Evading bot detection or fingerprinting; automating third-party sites without permission.
- Building the assistants themselves (the tool is called by them).
- Training or fine-tuning a model.

Milestone	Deliverable [**TODO:** agree with advisor]
M1 – MVP	Working demonstrator as above; this report
M2 – Internal evaluation	End-to-end system against this scope; second model; planted-instruction and vision runs at full N; live walk-through
M3 – External open house	Same build shown to an external audience [**TODO:** demo target]
M4 – Real-conditions validation	A real, permitted target used by at least one person outside the group; accessibility and edge-case review
M5 – Final defence and report	Final report with limitations and AI-use disclosure

7.3 Deliverables per milestone

8 AI tools used

[**TODO:** Required by the booklet: which tools, for what tasks, and roughly how much of the code and text they produced. Fill honestly; the viva checks that each member can explain and modify the components.] At runtime the system also calls a language model as a verified

Tool	Used for	Approx. share
[**TODO:** tool]	[**TODO:** code generation / debugging / documentation]	[**TODO:** %]
[**TODO:** tool]	[**TODO:** test and report generation]	[**TODO:** %]

candidate generator (Section 3); that is a product component, not development assistance. Analysis and conclusions in this report are the authors' own; [**TODO:** confirm].

9 Team contributions and advisor endorsement

[**TODO:** Who built what (intents and harness, model bridge, Android layer, evaluation and reporting).]
[**TODO:** Advisor endorsement date; the form asks whether it was obtained at least 24 hours before submission.]